

사용자 문제를 데이터로 정의하고, 엔지니어링과 **AI 모델**로 해결하는 소프트웨어 엔지니어 이찬호입니다



(+82) 010-2365-1837



ibear6954@gmail.com



github.com/teddygood



teddygood.github.io



linkedin.com/in/teddygood

About me



이찬호

CHANHO LEE

✉ ibear6954@gmail.com

☎ (+82) 010-2365-1837

🐙 GitHub 🌐 Blog  LinkedIn

Skills

- 언어: Python, SQL, C/C++, Java, JavaScript, TypeScript, CUDA, Rust
- 프론트엔드: React, Docusaurus
- 백엔드: Django, Flask, FastAPI, Node.js, Spring Boot, MySQL, MongoDB, PostgreSQL
- 클라우드/인프라: AWS, GCP, Docker, Kubernetes, Terraform, GitHub Actions
- 데이터/AI: Spark, Airflow, PyTorch, TensorFlow, LangChain, LangGraph, vLLM

Education

광운대학교 컴퓨터공학과 졸업

2018.03 - 2025.08

주요 수강 과목: 자료구조, 알고리즘, 데이터베이스, 운영체제, 컴퓨터구조, 컴퓨터네트워크, 시스템프로그래밍, 객체지향프로그래밍, 머신러닝, 확률과통계, 선형대수학, GPU 프로그래밍

Work Experience

AWS 클라우드 서포트 엔지니어 인턴

2024.12 - 2025.06

Strength

꾸준히 성장
함께 소통
끊임 없는 문제 해결

빠르게 변하는 기술 속에서도 배움을 멈추지 않고, 공부한 내용을 기록하고 나누며 한 단계씩 성장하려 합니다.
혼자 완벽한 답을 찾기보다 진행 상황과 고민을 투명하게 공유하고, 피드백을 반영해 더 나은 방향을 만듭니다.
문제를 회피하지 않고 먼저 정의한 뒤, 가설을 세우고 검증하며 실행 가능한 해결책으로 끝까지 연결합니다.

01

LLM PR Reviewer

2025.01.13 - 02.19 | AWS 인턴십 팀 프로젝트

프로젝트 설명

GitHub PR이 생성되면 diff를 분석해 LLM 리뷰 코멘트를 만들고 GitHub에 다시 남기는 서버리스 코드 리뷰 시스템입니다.

확인 포인트

PR diff 전처리, 검색 문맥 구성, Bedrock 프롬프트, 실패 알림을 연결해 워크숍에서 바로 운영 가능한 리뷰 흐름으로 만들었습니다.

LLM PR Reviewer

AWS 인턴십 팀 프로젝트 / Bedrock, Lambda, DynamoDB, OpenSearch, API Gateway, Python

상황	워크숍 참가자가 PR을 만들면 AI가 리뷰를 남기는 흐름이 필요했지만, PR diff는 여러 파일로 나뉘고 LLM 응답 품질도 쉽게 흔들렸습니다.
과제	PR 생성부터 diff 수집, 리뷰 가이드라인 검색, LLM 코멘트 생성, GitHub 반영까지 안정적으로 이어지는 서버리스 파이프라인이 필요했습니다.
행동	GitHub Webhook, API Gateway, Lambda, DynamoDB, S3, OpenSearch, Bedrock, GitHub API를 연결하고 multi-file diff 통합과 prompt 개선 루프를 설계했습니다.
결과	100개 이상의 테스트 PR로 품질을 점검했고, 45명 워크숍 참가자가 실습을 완주했으며 고객 만족도 4.92/5를 기록했습니다.
배운 점	LLM 시스템은 모델 호출보다 입력 구조화, 검색 기준, 평가 루프, 운영 알림이 함께 맞아야 서비스처럼 동작한다는 점을 배웠습니다.

LLM PR Reviewer

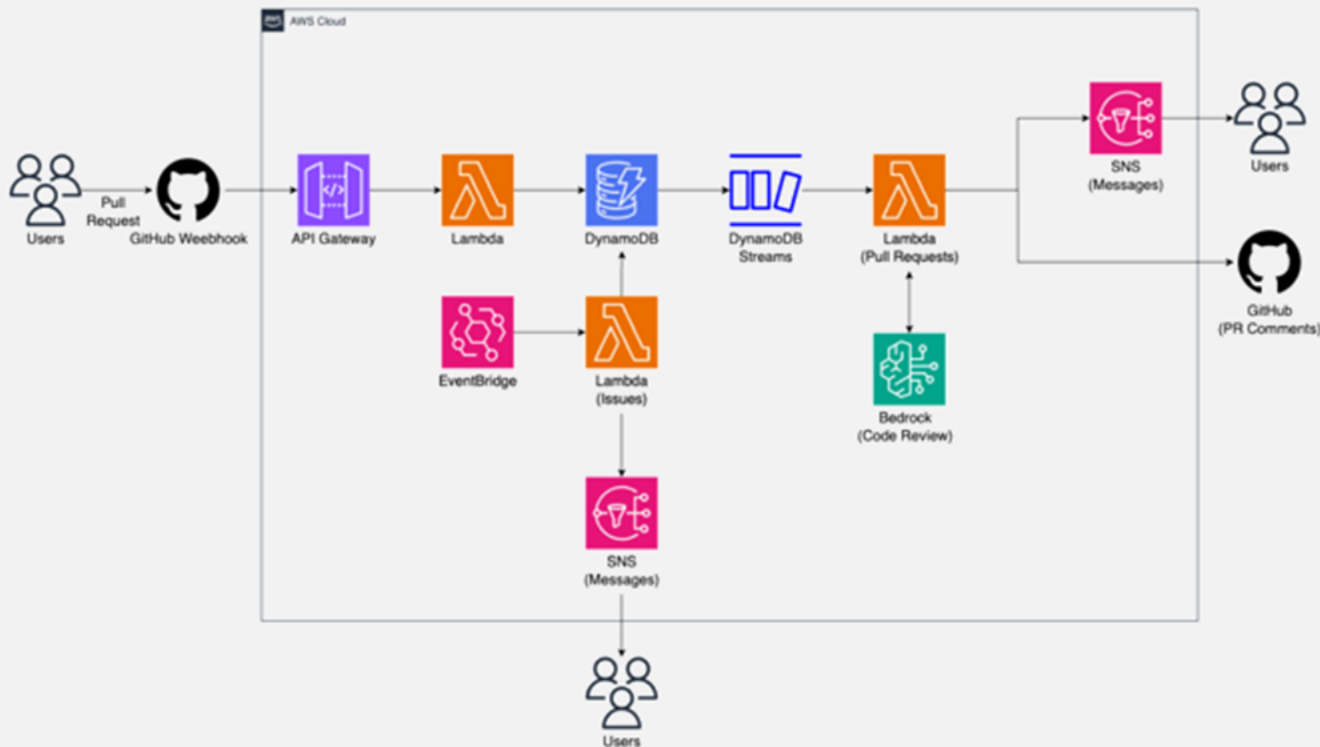
AWS 인턴십 팀 프로젝트 / Bedrock, Lambda, DynamoDB, OpenSearch, API Gateway, Python

프로젝트 개요

- GitHub PR을 입력으로 받아 LLM 리뷰 코멘트를 생성하고 GitHub에 다시 남기는 서버리스 코드 리뷰 시스템입니다.
- GitHub Webhook, Lambda, DynamoDB, OpenSearch, Bedrock, GitHub API를 연결해 PR 리뷰 흐름을 구성했습니다.
- 워크숍 운영을 위해 실패 알림, 재처리, test PR 기반 품질 확인을 함께 포함했습니다.
- 45명 실습 환경에서 참가자가 같은 흐름으로 리뷰 결과를 받을 수 있게 설계했습니다.

담당 역할

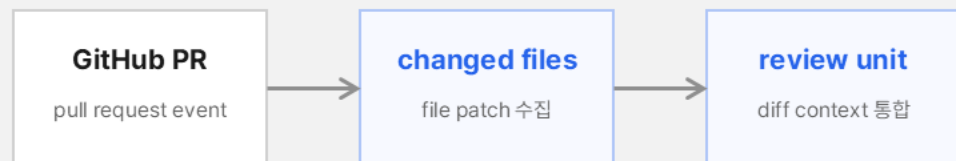
- PR diff를 파일 단위로 전처리하고 OpenSearch 검색 문맥과 Bedrock 프롬프트를 조합해 리뷰 입력을 안정화했습니다.
- DynamoDB Streams, Lambda, SNS로 비동기 처리와 실패 알림을 구성했습니다.
- 100개 이상의 test PR로 코멘트 품질을 점검하고, 워크숍 실습 흐름을 문서화했습니다.
- 참가자 PR 생성부터 리뷰 반영까지 end-to-end 운영 경로를 맡았습니다.



| Diff Context

여러 파일에 흩어진 PR 변경사항을 리뷰 가능한 입력으로 묶었습니다.

Multi-file diff 입력



파일 경계가 아니라 리뷰 의도 기준으로 prompt 입력을 구성

| 문제

- PR diff가 여러 파일로 나뉘면 변경 의도가 파일 경계 밖에 있었습니다.
- 그대로 넣으면 LLM이 파일별 단편 코멘트만 만들 위험이 있었습니다.
- 같은 PR에도 리뷰 기준이 흔들리면 워크숍 실습 품질이 낮아졌습니다.

| 시도

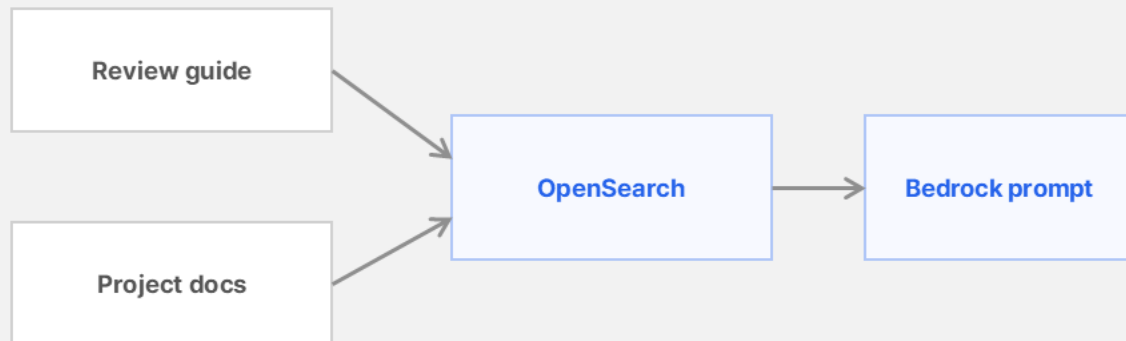
- GitHub API로 changed files와 patch를 모아 리뷰 단위로 재구성했습니다.
- 큰 diff는 핵심 변경, 파일 경로, 주변 문맥 중심으로 줄였습니다.
- Bedrock 입력 전에 체크리스트와 금지 응답 형식을 분리했습니다.

| 결과

- multi-file 변경을 하나의 리뷰 입력으로 안정화했습니다.
- 100개 이상의 test PR로 코멘트 누락과 반복 표현을 점검했습니다.
- 리뷰가 생성되는 이유를 운영자가 설명할 수 있는 입력 구조를 만들었습니다.

Review Context Retrieval

리뷰 기준과 과제 문맥을 검색해 Bedrock prompt에 붙였습니다.



문제

- diff만 넣으면 과제 기준, 리뷰 톤, 확인 범위가 매번 달라졌습니다.
- 참가자마다 같은 기준으로 피드백을 받아야 워크숍 운영이 가능했습니다.
- 리뷰 기준이 prompt 안에만 있으면 수정과 재사용이 어려웠습니다.

시도

- 리뷰 가이드라인과 프로젝트 문서를 OpenSearch 검색 문맥으로 붙였습니다.
- Bedrock prompt에서 코멘트 범위, 출력 형식, 예외 응답을 조정했습니다.
- 검색 실패 시에도 최소 리뷰 기준이 남도록 기본 문맥을 분리했습니다.

결과

- 참가자 PR에 기준이 흔들리지 않는 리뷰 코멘트를 생성했습니다.
- 45명 워크숍에서 실습 흐름을 완주하고 만족도 4.92/5를 기록했습니다.
- 모델 호출보다 검색 문맥과 출력 규칙이 품질에 중요하다는 점을 확인했습니다.

Workshop Operation

실패 알림과 재처리 흐름을 붙여 실습 중단을 줄였습니다.

45명

참가자

4.92/5

만족도

100+

test PR

DynamoDB

Streams 상태 변화

Lambda

실패 처리

SNS

운영 알림

문제

- 워크숍 중 LLM 호출이나 GitHub 반영이 실패하면 실습자가 바로 막힐 수 있었습니다.
- 운영자는 실패 지점을 빠르게 알아야 했습니다.
- 재시도 기준이 없으면 같은 PR을 수동으로 다시 처리해야 했습니다.

시도

- DynamoDB Streams, Lambda, SNS로 비동기 실패 처리와 알림을 연결했습니다.
- PR 상태 변화, 리뷰 생성, 알림 경로를 end-to-end로 테스트했습니다.
- 워크숍 전에 architecture와 실습 문서를 맞춰 운영 절차를 정리했습니다.

결과

- PR 생성부터 리뷰 반영, 실패 알림까지 흐름을 구성했습니다.
- 실습 중 문제 대응 시간을 줄일 수 있는 운영 기준을 만들었습니다.
- 45명 참가자가 같은 흐름으로 실습을 끝낼 수 있게 했습니다.

02

Recommendation System Backend

2026.01 – 2026.05 | 개인 프로젝트

프로젝트 설명

추천 시스템에 필요한 이벤트 수집, profile/feature 생성, 검색 저장소, reader API, 관측 지표를 묶은 백엔드 파이프라인입니다.

확인 포인트

RecSys Challenge 2025 Synerise behavioral dataset을 사용해 상품 행동 로그 기반의 이벤트 수집, 재처리, 조회 흐름을 검증했습니다.

Recommendation System Backend

개인 프로젝트 / Event Pipeline, Feature Store, Reader API / Go, Kafka, Flink, PostgreSQL, Elasticsearch, Grafana

상황	추천 시스템은 사용자 이벤트, 실험 배정, 지표 집계, 장애 관측이 분리되면 결과를 신뢰하기 어렵고 디버깅도 늦어집니다.
과제	추천 모델 이전에 이벤트를 안정적으로 저장하고, 실험 노출과 지표를 재현 가능하게 만들며, 장애 대응 시간을 줄이는 운영 구조가 필요했습니다.
행동	Kafka event pipeline, idempotent projection, deterministic assignment, SRM warning, Prometheus와 Grafana 모니터링을 나누어 구현했습니다.
결과	API 테스트와 Docker 실행 검증으로 주요 경로를 확인했고, 이벤트 수집부터 조회 API까지 데이터 흐름과 운영 지표를 포트폴리오 단위로 정리했습니다.
배운 점	추천 품질 이전에 이벤트 정의, 실험 기준, 관측 가능성이 맞아야 제품 판단이 가능하다는 점을 배웠습니다.

Recommendation System Backend

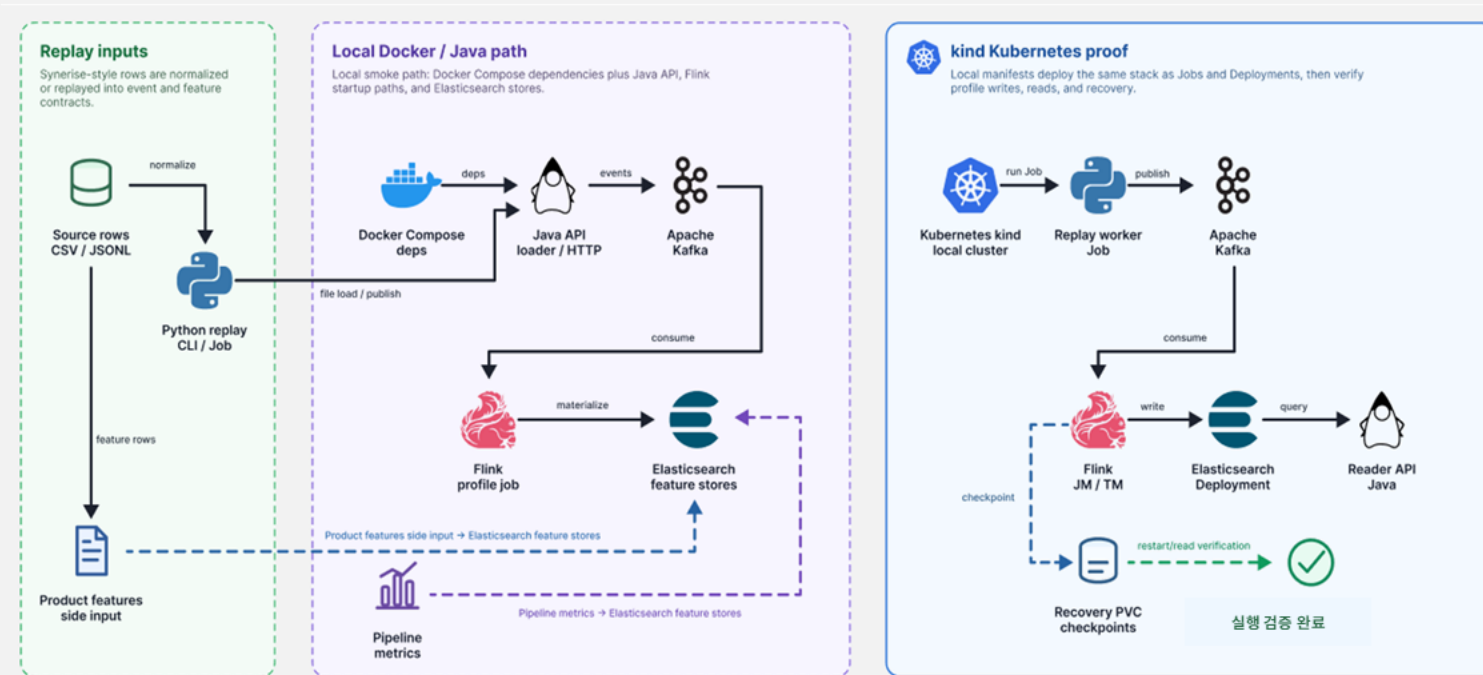
개인 프로젝트 / Event Pipeline, Feature Store, Reader API / Go, Kafka, Flink, PostgreSQL, Elasticsearch, Grafana

프로젝트 개요

- 추천 시스템에 필요한 이벤트 수집, 사용자 profile/feature 생성, 검색 인덱스, 조회 API를 하나의 백엔드 흐름으로 묶었습니다.
- RecSys Challenge 2025 Synerise behavioral dataset을 사용해 원본 행동 로그를 재생 가능한 이벤트로 바꿨습니다.
- Kafka 수집, Flink 집계, Elasticsearch 조회 역할을 분리했습니다.
- 로컬 실행과 Kubernetes 환경에서 재처리, checkpoint 복구, API 조회가 이어지는지 확인했습니다.

담당 역할

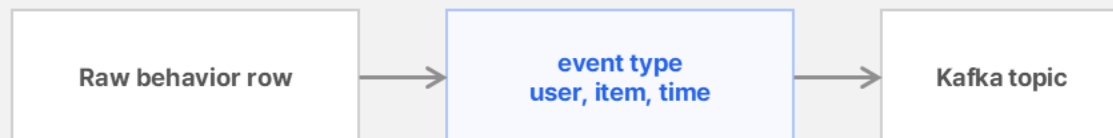
- CSV/JSONL 형태의 원본 행동 로그를 이벤트와 피처 생성 기준으로 정리하고, Python replay job과 Java loader 입력 경로를 설계했습니다.
- Kafka topic, Flink 집계, Elasticsearch mapping을 나누어 피처 생성, 조회, 재처리 책임을 분리했습니다.
- Docker Compose와 kind Job으로 실행해 checkpoint 복구와 reader API 조회까지 검증했습니다.
- Prometheus와 Grafana 기준으로 처리량과 장애 관측 지표를 분리했습니다.



| Event Normalization

RecSys Challenge 2025 Synerise behavioral dataset을 백엔드 이벤트 흐름으로 바꿨습니다.

데이터 흐름



Replay job으로 동일 로그를 다시 흘려보낼 수 있게 분리

| 문제

- 원본 행동 로그는 추천 API가 바로 쓰기 어려운 row 형태였습니다.
- 추천 품질을 말하기 전에 같은 이벤트를 다시 재생할 수 있어야 했습니다.
- 데이터 출처와 전처리 기준이 불명확하면 결과를 설명하기 어려웠습니다.

| 시도

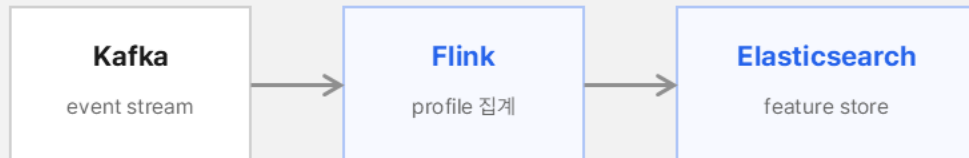
- RecSys Challenge 2025 Synerise behavioral dataset을 입력 데이터로 사용했습니다.
- row를 event type, user, item, timestamp 기준으로 정리했습니다.
- Kafka topic과 replay job 입력을 분리해 같은 로그를 반복 처리했습니다.

| 결과

- 사용한 데이터와 변환 기준을 포트폴리오 안에서 명확히 남겼습니다.
- raw row를 재생 가능한 event stream으로 바꿨습니다.
- 이후 Flink 집계와 API 조회가 같은 입력을 기준으로 동작하게 했습니다.

| Feature Materialization

Kafka 이벤트를 Flink 집계와 Elasticsearch 조회 구조로 연결했습니다.



checkpoint 복구와 재처리 기준 확인

| 문제

- 사용자 profile과 feature가 재처리 후에도 같은 기준으로 계산돼야 했습니다.
- 저장소와 조회 책임이 섞이면 장애 원인을 찾기 어려웠습니다.
- 추천 결과가 바뀌었을 때 집계 문제인지 조회 문제인지 나눠 볼 필요가 있었습니다.

| 시도

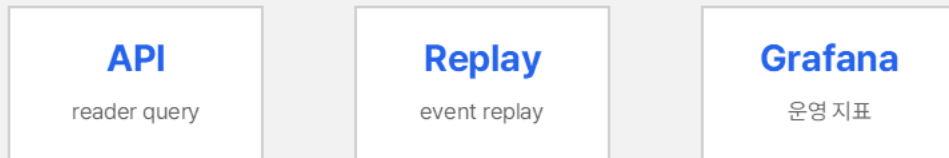
- Flink 집계, checkpoint, Elasticsearch mapping을 나누어 설계했습니다.
- feature 생성과 조회 책임을 reader API 앞에서 분리했습니다.
- 로컬 Docker와 kind 환경에서 재처리 경로를 따로 확인했습니다.

| 결과

- checkpoint 복구와 reader API 조회를 각각 검증했습니다.
- 추천 모델 이전의 데이터 신뢰도와 운영 가능성을 설명할 수 있게 됐습니다.
- 이벤트 흐름, 집계, 검색 저장소의 역할이 분명해졌습니다.

| Reader API and Monitoring

조회 API와 운영 지표를 함께 두어 추천 백엔드의 동작을 확인했습니다.



Prometheus와 Grafana로 처리량, 조회 흐름, 장애 상황을 관찰

| 문제

- 추천 결과가 이상할 때 API, 집계, 이벤트 중 어디가 문제인지 구분해야 했습니다.
- 로컬에서만 되는 검증으로는 운영 흐름을 설명하기 부족했습니다.
- 처리량, 지연, 조회 실패를 코드 밖에서 확인할 지표가 필요했습니다.

| 시도

- reader API 조회, replay, checkpoint 복구를 각각 검증했습니다.
- Prometheus와 Grafana로 처리량과 장애 지표를 분리했습니다.
- API 테스트와 실행 로그로 주요 경로를 반복 확인했습니다.

| 결과

- 이벤트 수집부터 조회 API까지 끊기지 않는 흐름을 확인했습니다.
- 단순 추천 모델이 아니라 운영 가능한 추천시스템 백엔드로 설명할 수 있게 됐습니다.
- 문제가 생겼을 때 어느 계층을 먼저 볼지 기준을 만들었습니다.

03

Optiver Trading at the Close

2023.09 - 2023.12 | Google Machine Learning Bootcamp

프로젝트 설명

Kaggle 장 마감 가격 예측 대회에서 auction/orderbook 데이터를 활용해 60초 뒤 가격 변화를 예측한 ML 프로젝트입니다.

확인 포인트

점수만 높이는 feature가 아니라 검증 구간 주변 날짜를 학습에서 제외하고 제출 시점의 inference cache까지 함께 설계했습니다.

Optiver Trading at the Close

Kaggle 대회 / LightGBM, feature engineering, time-series CV

상황	장 마감 경매 데이터에서는 체결, 호가, 불균형 신호가 빠르게 바뀌며 60초 뒤 가격 변화를 안정적으로 예측해야 했습니다.
과제	검증 구간 주변 날짜가 학습에 섞이지 않게 분리하면서 auction과 orderbook feature를 만들고, 스트리밍 추론 환경에서도 같은 feature를 유지해야 했습니다.
행동	124개 feature, Numba triplet imbalance, date_id 5-fold와 5-day purge, LightGBM ensemble, streaming API cache를 구성했습니다.
결과	private leaderboard 981 / 3,225 teams, score 5.4744를 기록했고, 최종 CV MAE는 5.8259였습니다.
배운 점	대회 성능은 feature 수보다 검증 설계와 온라인 추론 제약을 얼마나 일관되게 맞추는지가 중요하다는 점을 배웠습니다.

Optiver Trading at the Close

Kaggle 대회 / LightGBM, feature engineering, time-series CV

프로젝트 개요

- Optiver 장 마감 가격 예측 대회의 auction/orderbook 데이터를 활용한 ML 예측 프로젝트입니다.
- EDA, 124개 feature, date_id 기반 CV, LightGBM ensemble, streaming inference cache를 구성했습니다.
- 검증 구간 주변 날짜를 제외해 제출 성능을 과대평가하지 않도록 설계했습니다.
- 최종 제출 흐름까지 이어지는 검증 기준을 정리하고 private leaderboard 981 / 3,225를 기록했습니다.

담당 역할

- target, near_price, far_price, imbalance_size의 시간별 분포와 결측/이상 패턴을 EDA로 확인했습니다.
- 124개 feature를 만들고 date_id 기준 5-fold와 5-day purge로 검증 구간 주변 5일을 학습에서 제외했습니다.
- LightGBM ensemble과 streaming API cache를 구성해 학습, 검증, 제출 흐름을 끝까지 점검했습니다.
- zero sum, clipping 후처리까지 포함해 online inference 제약을 맞췄습니다.

EDA / Feature / CV / Inference

EDA

481일, 200개 종목, 523만 행 / 경매, 호가 스냅샷

Feature

124개 피쳐 / imbalance, spread, pair/triplet ratio, lag/diff

CV

date_id 5-fold + 5-day purge / 검증 구간 주변 5일 제외

Inference

최근 21행 rolling cache / 5-fold ensemble, zero_sum, clipping

Optiver 데이터와 검증 근거

데이터 규모, 결측 구조, fold별 검증 결과를 한 장에 정리했습니다. 55개 time bucket 단위로 관측했습니다.

ROWS

5,237,980

DATES

481

STOCKS

200

FEATURES

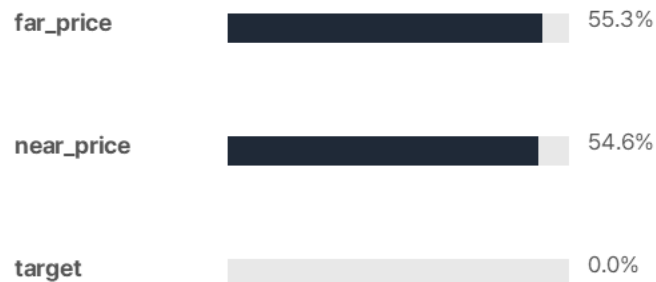
124

Target 분포



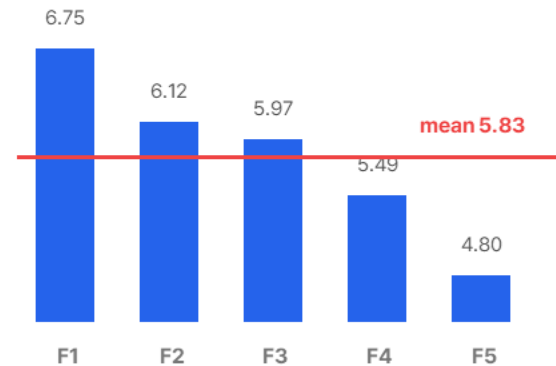
target은 중앙값이 0에 가깝지만 tail이 커서 MAE 기준 안정성이 중요했습니다.

결측/이상 패턴



far/near_price 결측은 단순 결함이 아니라 경매 단계별로 값이 생기는 데이터 구조였습니다.

date_id 기반 CV



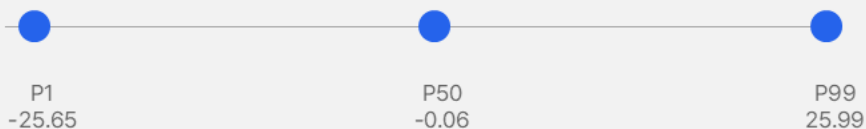
검증 구간 주변 5일을 학습에서 제외해 인접 날짜 패턴이 섞이는 위험을 줄였습니다.

요약: 단순 제출 코드가 아니라 데이터 구조 확인, 피쳐 생성, 날짜 기준 검증, 제출 API 추론 흐름까지 하나로 점검했습니다.

EDA and Target Shape

낮선 장 마감 경매 데이터를 먼저 구조와 분포로 이해했습니다.

Target 분포



far_price 결측 55.3%

near_price 결측 54.6%

문제

- closing auction 데이터는 가격, 호가, 불균형 신호가 시점별로 빠르게 변했습니다.
- target은 중앙에 몰렸지만 꼬리값이 커서 MAE 기준 안정성이 중요했습니다.
- far_price와 near_price 결측은 단순 결함으로 보기 어려웠습니다.

시도

- target, near_price, far_price, imbalance_size 분포와 결측을 먼저 확인했습니다.
- 종목과 날짜, time bucket별로 feature가 어떻게 달라지는지 나눠 봤습니다.
- 결측률과 target 분위수를 함께 보며 feature 후보의 위험을 줄였습니다.

결과

- 단순 모델링 전에 데이터 구조와 결측 패턴을 설명할 수 있게 됐습니다.
- 이후 feature와 검증 설계를 데이터 특성에 맞춰 잡았습니다.
- 경매 단계별로 생기는 값과 예측 타이밍을 구분해 볼 수 있었습니다.

| Validation Design

date_id 기준 fold와 purge로 가까운 날짜 패턴이 검증에 섞이는 위험을 줄였습니다.

date_id split



검증 주변 5일 제외 validation

5.8259

CV MAE

5-fold

date_id split

| 문제

- 가까운 날짜의 패턴이 학습과 검증에 동시에 들어가면 점수를 과대평가할 수 있었습니다.
- 제출 API에서는 학습 때와 같은 feature를 순차적으로 만들어야 했습니다.
- fold 성능만 보고 모델을 고르면 실제 제출에서 흔들릴 수 있었습니다.

| 시도

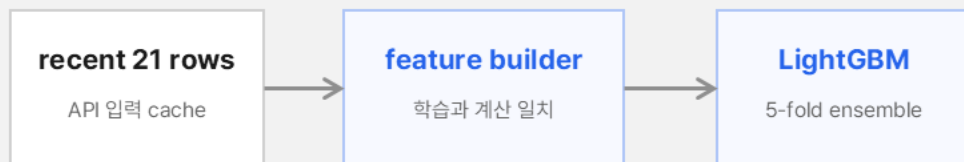
- date_id 5-fold와 5-day purge를 적용했습니다.
- fold별 결과를 비교해 특정 fold에만 의존하지 않는지 확인했습니다.
- CV 기준과 제출 API feature 생성 순서를 같이 점검했습니다.

| 결과

- 검증 구간 주변 5일을 제외해 인접 날짜 패턴이 섞이는 위험을 줄였습니다.
- private leaderboard 981 / 3,225, score 5.4744를 기록했습니다.
- 점수보다 검증 조건을 설명할 수 있는 기준을 남겼습니다.

Streaming Inference

제출 API에서 들어오는 row를 cache하고 학습 feature와 같은 계산 경로를 유지했습니다.



zero sum과 clipping으로 제출 조건에 맞게 후처리

124

features

21

cache rows

5.4744

score

문제

- notebook에서 만든 feature가 제출 API 환경에서도 같은 순서로 계산돼야 했습니다.
- 실시간 입력에서는 과거 row를 직접 다시 불러올 수 없어 cache가 필요했습니다.
- 후처리 기준이 흔들리면 ensemble 결과가 제출 형식과 맞지 않을 수 있었습니다.

시도

- 최근 21행 cache를 두고 lag/diff feature를 제출 시점에도 계산했습니다.
- 5-fold ensemble 뒤에 zero sum과 clipping 후처리를 적용했습니다.
- 학습 feature와 제출 feature가 같은 계산 경로를 지나도록 정리했습니다.

결과

- 학습, 검증, 제출 흐름을 하나의 파이프라인으로 설명할 수 있게 됐습니다.
- 대회 성능보다 검증 조건과 추론 제약을 함께 맞춘 경험을 남겼습니다.
- offline notebook과 online API 사이의 차이를 줄였습니다.

04

AI Shopping Assistant

2024.01 - 2024.02 | GDSC Solution Challenge

프로젝트 설명

시각장애 사용자가 상품 정보를 자연어로 질문하고
비교할 수 있도록 돕는 RAG 기반 쇼핑
어시스턴트입니다.

확인 포인트

상품 데이터 수집/정규화부터 벡터 검색, FastAPI
응답, TTS 친화 포맷까지 접근성 요구에 맞춰
연결했습니다.

AI Shopping Assistant

GDSC Solution Challenge / FastAPI, LangChain, ChromaDB, Cloud SQL, GCP

상황	시각장애 사용자가 상품 정보를 비교하려면 이미지와 복잡한 페이지 구조를 직접 탐색해야 해 접근성이 낮았습니다.
과제	쇼핑몰 데이터를 수집해 일관된 구조로 만들고, 자연어 질문에 상품 후보와 이유를 안정적으로 답하는 RAG 백엔드를 짧은 기간에 구현해야 했습니다.
행동	Selenium/BeautifulSoup 크롤링, Cloud SQL 적재, ChromaDB 임베딩 검색, LangChain QA chain, FastAPI API, Docker/GCP 배포를 구성했습니다.
결과	키워드 검색보다 자연어 기반 상품 질의응답이 가능해졌고, 프론트엔드 연동 전에도 Streamlit 프로토타입으로 흐름을 검증했습니다.
배운 점	챗봇 품질은 모델만이 아니라 데이터 수집 구조, 응답 포맷, 검색 기준, UI 피드백이 함께 맞아야 한다는 점을 배웠습니다.

AI Shopping Assistant

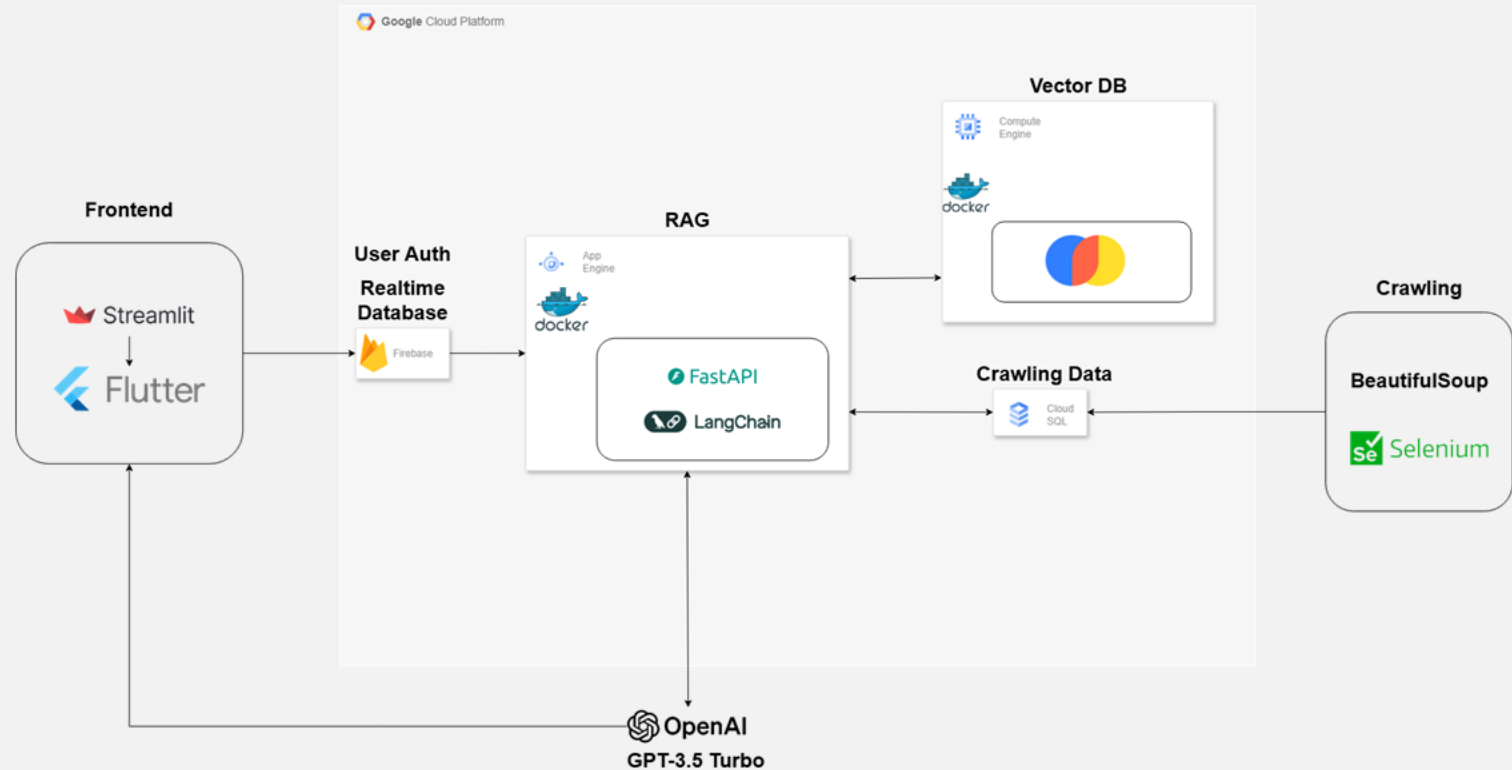
GDSC Solution Challenge / FastAPI, LangChain, ChromaDB, Cloud SQL, GCP

프로젝트 개요

- 시각장애 사용자의 상품 탐색을 돕는 RAG 기반 쇼핑 질의응답 백엔드입니다.
- 쇼핑몰 크롤링 데이터를 Cloud SQL과 ChromaDB로 나누고 LangChain, FastAPI로 질의응답 API를 구성했습니다.
- 상품명, 가격, 옵션, 설명을 검색 가능한 구조로 정리해 자연어 조건과 연결했습니다.
- 프론트엔드 연동 전 Streamlit과 Docker/GCP 배포로 전체 응답 흐름을 확인했습니다.

담당 역할

- 백엔드와 RAG 파이프라인 담당으로 Selenium/BeautifulSoup batch, Cloud SQL schema, Chroma embedding index를 설계했습니다.
- ko-sroberta-multitask 임베딩과 Chroma를 사용해 비용과 운영 부담을 줄이고, 데이터 규모에 맞는 검색 구조를 선택했습니다.
- FastAPI 응답에서 후보, 가격, 추천 이유가 함께 반환되도록 응답 구조를 정리했습니다.
- Streamlit 테스트 UI로 프론트엔드 연동 전에도 질문, 검색, 답변 흐름을 확인했습니다.



Data Collection

시각장애 사용자가 비교 가능한 상품 정보를 얻도록 쇼핑몰 페이지를 구조화했습니다.

수집 필드 정리

상품명	가격	옵션	설명
-----	----	----	----

Selenium과 BeautifulSoup으로 페이지별 위치 차이를 흡수

Cloud SQL schema로 저장해 검색 전 데이터 기준을 고정

문제

- 이미지와 배너에 핵심 정보가 있어 텍스트 비교가 어려웠습니다.
- 상품명, 옵션, 가격 위치가 사이트마다 달라 수집 기준이 흔들렸습니다.
- 상품 정보가 빠져도 사용자가 바로 알아차리기 어려웠습니다.

시도

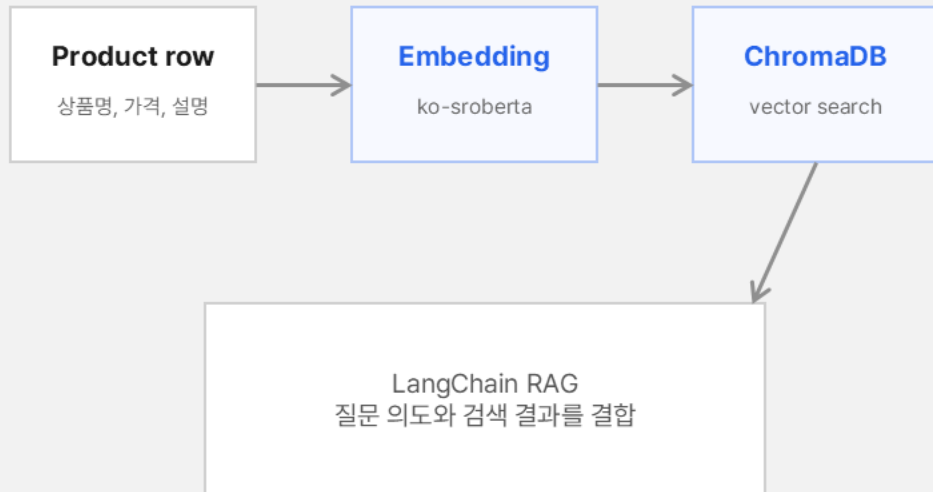
- 크롤러에서 상품 필드를 먼저 정하고, 누락 필드는 저장 전 검증했습니다.
- Cloud SQL에 상품 데이터를 일관된 schema로 저장했습니다.
- 수집, 정제, 저장 단계를 분리해 검색 전 데이터 품질을 점검했습니다.

결과

- RAG 검색에 사용할 상품 후보 데이터를 안정적으로 만들었습니다.
- 프론트엔드 없이도 데이터 수집과 저장 흐름을 독립적으로 검증했습니다.
- 접근성 문제를 모델 이전의 데이터 구조 문제로 바꿔 다뤘습니다.

Retrieval Pipeline

상품 설명과 사용자 조건을 ChromaDB 검색과 LangChain RAG로 연결했습니다.



문제

- 키워드 검색만으로는 사용자가 말한 조건을 상품 특징과 연결하기 어려웠습니다.
- 추천 이유가 없으면 접근성 개선 효과도 설명하기 어려웠습니다.
- 상품 후보가 많아질수록 검색 기준과 답변 근거를 분리해야 했습니다.

시도

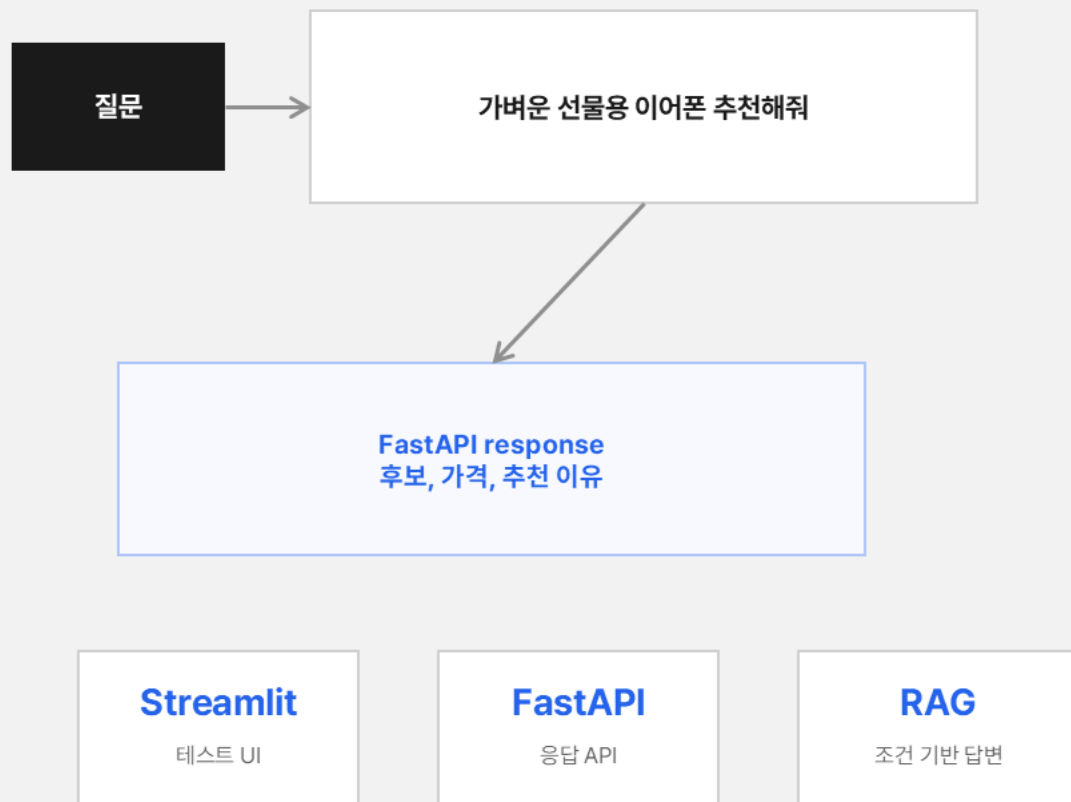
- 상품 설명을 embedding하고 ChromaDB에서 후보를 검색했습니다.
- LangChain RAG로 질문 의도, 상품 후보, 응답 포맷을 묶었습니다.
- 검색 결과와 답변 문장을 나눠 후보 누락과 과장 응답을 줄였습니다.

결과

- 자연어 조건에 맞는 상품 후보와 이유를 함께 반환할 수 있게 했습니다.
- 모델 자체보다 검색 기준과 응답 구조가 품질에 중요하다는 점을 확인했습니다.
- 사용자가 비교 기준을 다시 찾지 않아도 되는 응답 흐름을 만들었습니다.

| Answer API

FastAPI와 Streamlit으로 실제 질문에서 후보와 이유가 이어지는지 확인했습니다.



| 문제

- API 응답이 상품 후보만 주면 사용자가 비교 기준을 다시 찾아야 했습니다.
- 프론트엔드 완성 전에도 전체 응답 흐름을 확인할 수 있어야 했습니다.
- 백엔드와 RAG 파이프라인 책임이 섞이면 팀 내 연동이 늦어질 수 있었습니다.

| 시도

- FastAPI에서 질문, 검색, 답변 생성을 하나의 응답 경로로 분리했습니다.
- Streamlit으로 실제 질문을 넣어 프론트 연동 전 흐름을 점검했습니다.
- Docker/GCP 배포 전후로 동일한 질문에 대한 응답 구조를 확인했습니다.

| 결과

- 후보와 추천 이유를 함께 주는 쇼핑 질의응답 흐름을 만들었습니다.
- 팀 프로젝트에서 백엔드와 RAG 파이프라인 역할을 분명히 맡았습니다.
- 화면이 없어도 API 응답만으로 서비스 흐름을 검증할 수 있게 했습니다.

05

Pyodide

2025.07 – 2026.03 | 2025 오픈소스 컨트리뷰션 아카데미

프로젝트 설명

Python 과학 계산 스택을 브라우저 WebAssembly 환경에서 더 빠르고 안전하게 실행하기 위한 오픈소스 기여입니다.

확인 포인트

테스트, benchmark, 빌드 옵션 변경을 리뷰 가능한 단위로 정리했고, Pyodide 0.29 릴리즈 노트에 기여 내용이 기록됐습니다.

Pyodide WebAssembly SIMD

오픈소스 기여 / WebAssembly, C, Python, OpenBLAS

상황 Pyodide에는 WASM SIMD 지원과 OpenBLAS SIMD 빌드가 실제 패키지와 CI에서 어떻게 검증되는지 확인할 기준이 부족했습니다.

과제 SIMD intrinsics가 동작하는지, OpenBLAS SIMD를 기본값으로 둘 만큼 유효한지 재현 가능한 방식으로 판단해야 했습니다.

행동 test-simd 패키지, scalar 비교 테스트, -msimd128 OpenBLAS 빌드, warm-up을 반영한 벤치마크, scipy 테스트를 정리했습니다.

결과 SIMD 테스트 패키지와 OpenBLAS SIMD 기본 빌드 관련 PR이 병합됐고, pyodide-build cleanup CLI 후속 PR도 병합됐습니다.

배운 점 성능 개선은 빠른 수치보다 측정 조건, 워밍업, 기본값 변경의 유지보수 비용을 함께 설득해야 한다는 점을 배웠습니다.

Pyodide

오픈소스 기여 / WebAssembly, C, Python, OpenBLAS

프로젝트 개요

- Pyodide의 WebAssembly 환경에서 SIMD와 OpenBLAS 성능 개선을 검증한 오픈소스 기여입니다.
- test-simd 패키지, scalar 비교, OpenBLAS -msimd128 빌드, benchmark recipe로 변경 근거를 남겼습니다.
- SIMD 검증 기여가 Pyodide 0.29 release note의 SIMD Support Exploration 항목에 contributor로 소개되었습니다.
- pyodide와 pyodide-build PR을 연결해 테스트, 벤치마크, cleanup CLI까지 이어지는 기여로 정리했습니다.

담당 역할

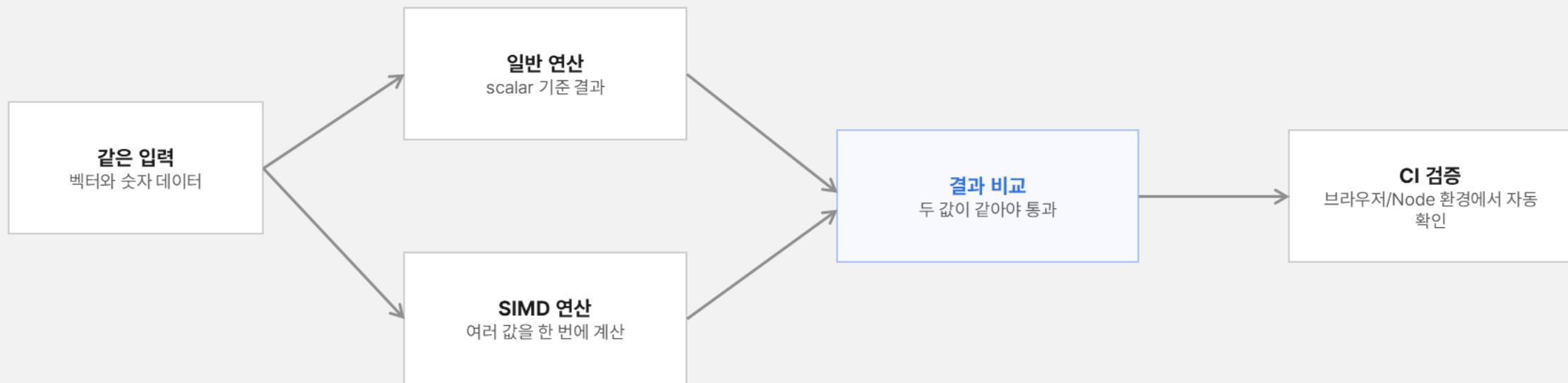
- test-simd 패키지에 SIMD/scalar 결과 비교를 추가하고 warm-up과 측정 조건을 분리한 benchmark recipe를 정리했습니다.
- macOS ARM, GCC flag, OpenBLAS 기본값 변경 등 리뷰 이슈를 재현해 수정했습니다.
- pyodide PR과 pyodide-build 후속 PR을 연결해 테스트와 cleanup CLI까지 이어지게 했습니다.
- 성능 수치보다 재현 가능한 조건과 기본값 변경의 유지보수 비용을 함께 설명했습니다.

Pyodide 기여 흐름 요약



SIMD 검증을 쉽게 보면

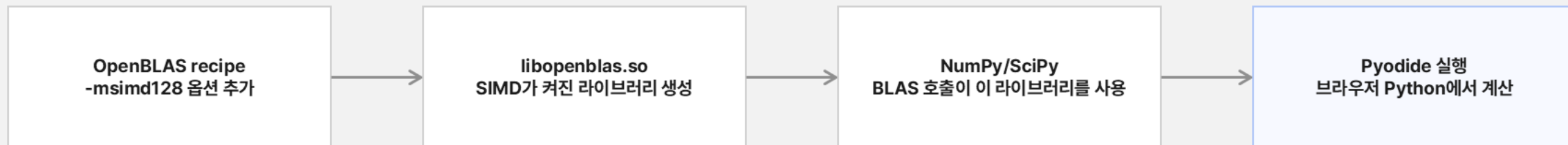
같은 계산을 일반 경로와 SIMD 경로로 실행해 결과가 같은지 확인하는 테스트



왜 중요한가
계산이 빠르더라도 결과가 틀리면 성능 개선이 아니라 계산 버그입니다.

OpenBLAS SIMD가 적용되는 위치

빌드 옵션 하나가 Pyodide 사용자의 NumPy/SciPy 계산 경로로 이어지는 구조



검증

1

기존 테스트

패키지가 깨지지 않는지 확인

2

벤치마크

sdot, sgemm 계산 시간 비교

3

최종 선택

O3 제외, SIMD non-O3만 기본값 유지

벤치마크를 설득력 있게 만든 과정

리뷰어가 믿을 수 있도록 측정 조건을 정리하고 최종 기본값을 좁힌 흐름



수정 이유

1

숫자를 의심

결과가 이상하면 원인부터 확인

2

조건을 고침

측정 방식과 재현 조건 명확화

3

변경을 줄임

저장소에 남길 최소 기본값 선택

작은 CLI 기여를 설계한 방식

사용자가 직접 쓰는 명령어는 내부 helper를 먼저 안전하게 만든 뒤 열었습니다



왜 의미 있나

1

안전한 기본값

중요 산출물을 기본으로 지우지 않음

2

테스트 포함

패키지와 태그 선택 경로 확인

3

리뷰 반영

명령 이름과 책임 범위 조정

감사합니다